



**BITS Pilani**  
K K Birla Goa Campus

# PRINCIPLES OF PROGRAMMING LANGUAGES

## PoPL: Preparing for Compiler Construction

**RAMPRASAD S. JOSHI**

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION SYSTEMS,  
BITS PILANI, K K BIRLA GOA CAMPUS

# The Beginning of the Beginning



A new tanker arrives on site in Qatar:  
The newly appointed American Manager  
tells the local supervisor to ensure that  
the tanker is clearly labeled:  
“Diesel Fuel” in Arabic and  
“No Smoking” in Arabic  
This is what he got.

Q. How to write a functional program in OCaml?

A. `let output = "a functional program" ;;`



As described in <https://www.bits-pilani.ac.in/wp-content/uploads/Bulletin-2023-24.pdf> page VI-38, the course covers features of programming languages and introduces the main programming paradigms. It covers, at various levels of detail,

- ▶ Semantics and features of programming languages :-
  - ▶ Control Abstraction,
  - ▶ Data Types and Data Abstraction,
  - ▶ Scope and Parameter passing and
  - ▶ Concurrency related features.
- ▶ Various aspects of runtime environments like
  - ▶ global and local data,
  - ▶ code,
  - ▶ function call stacks,
  - ▶ dynamically allocated data,
  - ▶ runtime features for exceptions and
  - ▶ threads.



- ▶ Introduction to programming paradigms :-
  - ▶ Functional paradigm – formal elements of lambda calculus,
  - ▶ introduction to syntax of common functional programming languages
  - ▶ programming exercises that explore the functional paradigm
- ▶ Logic programming paradigm :- formal elements of logic programming and programming tasks that explore the logic paradigm.
- ▶ Scripting as a paradigm.
- ▶ Domain specific languages.
- ▶ Applications of the principles of programming languages – program verification, software testing and security.

# Presentation ToC I



## Preliminaries

- Languages and Abstraction Levels
- Abstraction

## Programming Language Design

- The Clash of Purists and Linguists
- Lessons for Us

## Imperative Paradigm

- Structured Programming
- Principles of Imperative Languages
- Data Types

## Procedural to OO

## Object-Orientation

## Procedure Activation

## OOP in Java

## Generics

## References

# Section



## Preliminaries

- Languages and Abstraction Levels
- Abstraction

## Programming Language Design

- The Clash of Purists and Linguists
- Lessons for Us

## Imperative Paradigm

- Structured Programming
- Principles of Imperative Languages
- Data Types

## Procedural to OO

## Object-Orientation

## Procedure Activation

## OOP in Java

## Generics

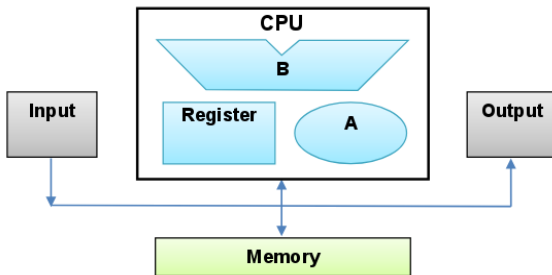
## References



# All Computation Is ...



- ▶ Built on few primitives
- ▶ From a static foundation (can be small or large, but **finite**)
- ▶ Using few elementary operations composed into infinite permutations and combinations
- ▶ The *von Neumann Machine*



# Higher Level Languages as Abstractions



- ▶ The “height” of a language is its closeness to human “level”—that means, its expressive power.
- ▶ The language should let us express our intention;
- ▶ and provide a toolchain to turn that intention into computation;
- ▶ with the least burden on the programmer for the “how” part;
- ▶ and with as much fidelity—actual computation effected should realise the original intention—as possible.

This view calls natural-language-like abstraction as the highest level and machine code as the lowest.



- ▶ Formal languages are mathematical constructions (sets of strings, defined by some structure)



- ▶ Formal languages are mathematical constructions (sets of strings, defined by some structure)
- ▶ Infinite universes of logical expressions (propositions, predicates) make formal languages



- ▶ Formal languages are mathematical constructions (sets of strings, defined by some structure)
- ▶ Infinite universes of logical expressions (propositions, predicates) make formal languages
- ▶ Discrete structures, number systems (**N**, **Q**, **Z** and groups and rings and fields and algebras over them) are more such examples.



- ▶ Formal languages are mathematical constructions (sets of strings, defined by some structure)
- ▶ Infinite universes of logical expressions (propositions, predicates) make formal languages
- ▶ Discrete structures, number systems (**N**, **Q**, **Z** and groups and rings and fields and algebras over them) are more such examples.
- ▶ If we take only the set of “valid” programs in a programming language as a language, it is a formal language



- ▶ Formal languages are mathematical constructions (sets of strings, defined by some structure)
- ▶ Infinite universes of logical expressions (propositions, predicates) make formal languages
- ▶ Discrete structures, number systems (**N**, **Q**, **Z** and groups and rings and fields and algebras over them) are more such examples.
- ▶ If we take only the set of “valid” programs in a programming language as a language, it is a formal language
- ▶ However, with its semantics, a programming language is much more than a formal language.

# Programming Languages Are As Human as English



- ▶ Most languages are discrete and linear (1-D)

# Programming Languages Are As Human as English



- ▶ Most languages are discrete and linear (1-D)
- ▶ But the universe that they model (and we otherwise perceive) is neither discrete nor finite dimensional

# Programming Languages Are As Human as English



- ▶ Most languages are discrete and linear (1-D)
- ▶ But the universe that they model (and we otherwise perceive) is neither discrete nor finite dimensional
- ▶ Thus, we use languages (formal, natural, programming) to condense the unfathomable possibilities of the universe into reproducible, verifiable and uniquely identifiable representations.



It is not our wont to characterise abstraction in abstract terms. We can only give concrete examples of abstraction of ideas from concrete experience.



Content is concrete, form is abstract. That's why, though we find scarcely a perfect circle or ball, we call everything from a cylinder to a rectangle with angular corners “round”—even persons with shoulder width comparable to their height are called “round”.

- ▶ The name of a library function, or its signature; a class in a standard library, etc. are abstractions of the computations they promise.
- ▶ Real numbers are abstract, floating point computations are concrete.
- ▶ Placement tests and interviews on OOP are abstract, an OO program designed and developed by a real programmer is concrete.



- ▶ The following hierarchy of abstractions is obvious:

$\alpha\beta\gamma\sum$  Mathematical formulation of GCD of any two members of a ring equipped with unique factorization

- = The identities

$$\gcd(a, b) = \gcd(b, a) = \gcd(|a - b|, b) = \gcd(a, |a - b|)$$

10 15 5  
19 56 1  
304 512 128

A 3-column matrix or table of two arguments and their GCD in each row.

- Euclid's algorithm using subtraction

% Euclid's algorithm using division-remainder

C One or more C program(s) to compute the GCD of integers, or polynomials, or quadratic integers, etc.

- ▶ A binary executable of such a program



Scott [2015, Chapter 3] says:

We call programming languages (except assembly languages) as “higher” level in two senses of the word abstraction:

- ▶ Abstraction of the programmable machine. A microprocessor with the OS, a JVM or JRE, etc. are examples of the concrete machines that are abstracted by C/C++/Java/Python etc.
- ▶ Naming pieces of code to express the purpose or function or role (the “what”) of that piece, hiding the details of the “how” part.

Thus,

- ▶ Subroutines (functions in C/C++, methods in Java, callable procedures in general) are *control abstractions*.
- ▶ Classes are *data abstractions*.

On these ideas, we further dissect the concrete from its abstract representation in the rest of this section.



- ▶ The *von Neumann* machine is *mechanical* computation. Mechanical computation means something we can emulate exactly (albeit extremely slowly) using coarse, macro level devices like flipping switches: clearly discernible elements of the interaction of which is broken into steps that we can carry out without using any discretion or horse sense.
- ▶ For example, we can demonstrate any sorting algorithm on a bunch of books as if the bunch is an array. We can use graph algorithms for the treasure hunt game in the Campus. If we switch lamps on doors in a maze selectively on and off, we can guide a live mouse trained with those lights to follow a maze navigation algorithm exactly.
- ▶ Industrial robots doing repetitive tasks by limbs with limited degree of freedom are based on mechanical computation – controlled by Programmable Logic Controllers programmed using Ladder Diagrams.
- ▶ Quantum computation is not mechanical in this classical sense. Semi-autonomous robots are examples of a combination of mechanical computation and human discretion.

# What is Not Programmable?



- ▶ What is not finitely expressible is not programmable. Like, irrational numbers that are not roots of polynomials with integer coefficients. Thus, most reals are not programmable. Equality of arbitrary reals is not decidable.
- ▶ When all the possible situations are not finitely expressible, autonomous control is not programmable. Like, the angle-of-attack control during flight take-off.
- ▶ Captchas

Tea



Sugar



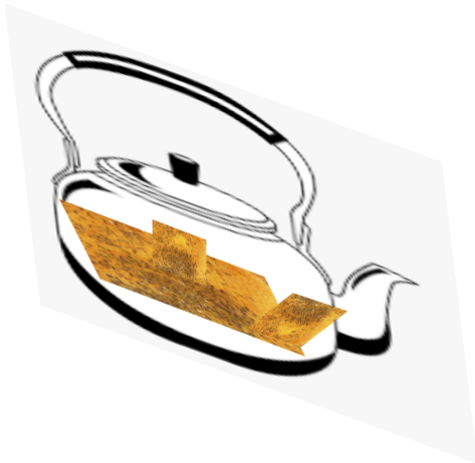
Milk, Water



Input: Ingredients are put into the "hardware"



Processing: They are boiled together.



Output: Tea is served.



Output: Tea is served.

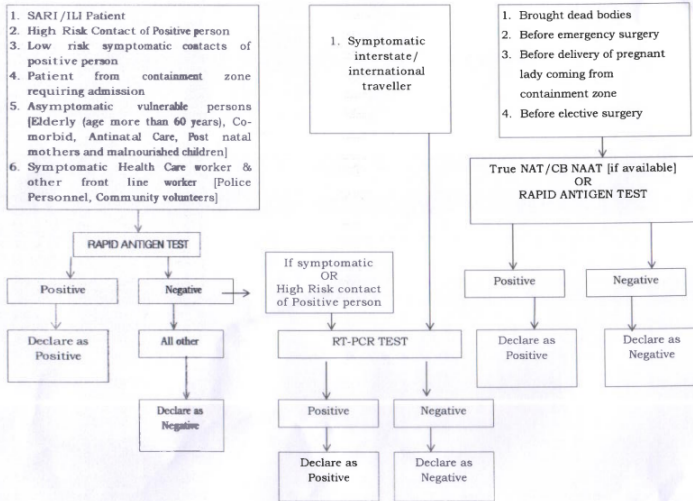
Not “programmable” like bakes in a microwave?



How come this is readily “programmable”?

*Hint: Can you model the operation of a tea/coffee vending machine by a finite automaton?*

## COVID 19 : TESTING PROTOCOL



Can this handle the uncertainty, reversibility and diversity in the biosphere?

# Section



## Preliminaries

- Languages and Abstraction Levels
- Abstraction

## Programming Language Design

- The Clash of Purists and Linguists
- Lessons for Us

## Imperative Paradigm

- Structured Programming
- Principles of Imperative Languages
- Data Types

## Procedural to OO

## Object-Orientation

## Procedure Activation

## OOP in Java

## Generics

## References





Reading, Speaking, Writing, Listening skills in a Programming Language

Remember: Programming languages (or, languages, of any kind) are for humans, not for machines.

Again, Scott [2015, Chapter 3] says that ease of programming is a stronger motivation that has come to replace portability (“machine independence”) in programming language design. (If we see closely, the desire for independence of programming from hardware and system–machine–characteristics also arises from the desire for ease of use.)



Ease of use includes

- ▶ Ease of designing, developing, testing programs
- ▶ Ease of modelling, simulation and formal verification of programs
- ▶ Ease of deployment of programs
- ▶ Ease of validation of programs in application settings
- ▶ Ease of maintenance, troubleshooting, system-level repairs, updates and upgradation
- ▶ Ease of documentation and training in deployment of programs
- ▶ Ease of delegation, sharing, arguments and explanations about programs

# What is Programming?



## Programmability

- ▶ What is programmable?
- ▶ In which ways is it programmable?
- ▶ In which ways is it *not* programmable?

## Three Essentials

**Elementary Operations** Arithmetic, Logical operations. Or, Move the cursor, point to an icon, click, etc.

**Sequencing** (*Note: No timing*) Which operations follow which other operations, when, how. Click or double click? Cursor pointer or hand tool? Complication: Even callbacks are real time?

**Repetition** Recursion, iteration, cycling (okay, what about sustained keypresses?), etc.



- ▶ In a shell, we type commands and (using `readline`) sometimes reuse them: each program is static and computation dynamic. e.g. Running the same command again after a `cd` change of directory may give different results.
- ▶ We can program the shell using a script. But the dynamism in the computation remains.
- ▶ We can pass arguments to the shell script to handle this dynamism.
- ▶ Still, the environment in a shell remains dynamic, whatever we do to program it.
- ▶ What is the environment of a C program?



THE ARROGANT PROGRAMMER: DIJKSTRA AND  
WEGNER CONSIDERED HARMFUL

by

Robert J. Du Works

Stephen W. Smoliar

TECHNION

Haifa-Israel

(Ed.: I think this article should definitely spur further comments on the Kandel-Wegner "debate" on the proper path for Computer Science Education to follow. (Comm. of ACM, June, 1972.) This debate has led to a great deal of discussion of many aspects of Computer Science Education. I think this is a sign that perhaps we are maturing as a discipline. I hope this article adds more "fuel to the fire," and that many of you will react to the comments in this paper. The more debate we have, the better are our chances to sift out what we feel are the most important aspects of Computer Science Education. Though agreement on these "basic principles" might not be forthcoming for awhile, I think articles and discussions on this topic are crucial to our development. So please take the time to write me a letter elaborating on your particular philosophy.)

## The Humble Programmer

by Edsger W. Dijkstra



As a result of a long sequence of coincidences I entered the programming profession officially on the first spring morning of 1952, and as far as I have been able to trace, I was the first Dutchman to do so in my country. In retrospect the most amazing thing is the slowness with which, at least in my part of the world, the programming profession emerged, a slowness which is now hard to believe. But I am grateful for two vivid recollections from that period that establish that slowness beyond any doubt.

I was supposed to study theoretical physics at the University of Leiden simultaneously, and as I found the two activities harder and harder to combine, I had to make up my mind, either to stop programming and become a real, respectable theoretical physicist, or to carry my study of physics to a formal completion only, with a minimum of effort, and to become . . . , yes what? A programmer? But was that a respectable profession? After all, what was programming? Where was the sound body of knowledge that could sup-

## The Humble Programmer

by Edsger W. Dijkstra



As a result of a long sequence of coincidences I entered the programming profession officially on the first spring morning of 1952, and as far as I have been able to trace, I was the first Dutchman to do so in my country. In retrospect the most amazing thing is the slowness with which, at least in my part of the world, the programming profession emerged, a slowness which is now hard to believe. But I am grateful for two vivid recollections from that period that establish that slowness beyond any doubt.

I was supposed to study theoretical physics at the University of Leiden simultaneously, and as I found the two activities harder and harder to combine, I had to make up my mind, either to stop programming and become a real, respectable theoretical physicist, or to carry my study of physics to a formal completion only, with a minimum of effort, and to become . . . , yes what? A programmer? But was that a respectable profession? After all, what was programming? Where was the sound body of knowledge that could sup-



# What's the Debate About? I



Long before he got the Turing Award, Dijkstra decried the use of `gotos` in structured programming in Dijkstra [1968] and Rice and Dijkstra [1968]. In his Turing Award acceptance speech, Dijkstra said, *inter alia*, that

- ▶ I/O interrupts, occurring at unpredictable and irreproducible moments, compared with the old sequential machine that pretended to be a fully deterministic automaton, has been a dramatic change, creating grave logical problems.
- ▶ Secondly, multilevel stores require extensive documentation that still cannot solve the elusive complications of managing them.
- ▶ Above all, the increased power of hardware, together with the perhaps even more dramatic increase in its reliability, made solutions feasible that the programmer had not dared to dream about a few years before.

# What's the Debate About? II



- ▶ Programs—even alternative programs for the same task and with the same mathematical content—can differ tremendously in their intellectual manageability.
- ▶ A number of rules have been discovered, necessary for sustaining intellectual manageability of programs.
- ▶ These rules are of two kinds. Those of the first kind are easily imposed by the choice of a programming paradigm and a compiler. Examples are the exclusion of goto-statements and of procedures with more than one output parameter.
- ▶ The second kind are elements of discipline required from the programmer. Algol-60 was [to be] designed so as to enforce this discipline at the outset.



The last point in the summary of Dijkstra's speech we cited above is the take-off point for Dijkstra's book Dijkstra [1976] (<https://seriouscomputerist.atariverse.com/media/pdf/book/Discipline%20of%20Programming.pdf>). Though Dijkstra titled the book with "A" Discipline, Prof. David GriesGries [1981] completed his "project" by offering "The" Science (<https://seriouscomputerist.atariverse.com/media/pdf/book/Science%20of%20Programming.pdf>).



- ▶ Computer science, as a “discipline”—as in, ‘*What are computer scientists like? Craftspeople, artistes, or philosophers? What if their services or products result in a disaster for those who depend on them?*’—was ill-defined in 1972, troubling the administrators, academicians, and professionals (like the ACM) alike.
- ▶ Prof. Kandel, in the CS Dept of New Mexico Institute of Mining and Technology, proposed this:

Therefore, it is recommended that design projects, such as the minicomputer UHC-1, from basic specifications to complete circuitry and machine language, be performed in a senior-level course on computer design in order to stimulate student interest in computer-based industries, to increase communication between groups of students on a design team (after all, industry is based on a team effort), and to expose the students to real-life constraints (cost is “sometimes” a prime consideration).



- ▶ His recommendation was based on

Even though much of the complexity of modern computer systems is due to the need for efficiency in the design, we should teach the prospective designer not to worry about how well the computational system works before determining whether it will work at all. Then, consideration of efficiency and other real-life constraints (cost, marketing, etc.) must be emphasized until a real-life design is achieved by the gradual removal of the unrealistic assumptions which build the first layer in the student's studies. In order to reinforce this development and to lend credibility to the system design (hardware or software), the student's work must be complemented by a practical, carefully-designed case study. One attempt to go quite far in this direction



Prof. Howard Tompkins of CS Dept in Indiana University of Pennsylvania said (letter to editor, Lynn [1972]):

*A Computer Science major today, whether at bachelor or master level, is being prepared not just to program, or to analyze real world problems in preparation for programming, but rather:*

- *to improve the tools that programmers and systems analysts use,*
- *to develop better machine systems, better software systems, better languages for communicating with the machines, and better methods for solving intractable problems,*
- *to manage activities that are strongly computer dependent,*
- *to teach about computers at college level, or*
- *to advance the fundamental theory of digital information processors.*

*Some graduates of Computer Science programs may be programmers or systems analysts, but they will have changed direction, or missed their aim, or simply still be in their learning phase.*

# Okay, “Practitioners” Respond



## DuWorks and Smoliar [1972]:

One thing is certain: science has arisen from man's quest to formulate terse generalizations of his myriad observations. What is important is that observation precedes generalization—preferably by as great a distance as possible. As Suzanne (Langer) has written:

*General theories should be constructed by generalization from the principles of a special field, known and understood in full detail. Where no such systematic order exists to serve as a pattern, a general theory is more likely to consist of vague generalities than of valid generalizations.*

- ▶ “Cart before the Horse” : Du Works and Smoliar thought (in 1972) that *Dijkstra's “formal” foundations of theory constitute a clear-cut cart-before-the-horse case of observation-generalization inversion.*
- ▶ ALGOL-60 v/s PL/I : ALGOL-60's designers' insistence on mathematical purity was “formal overkill” causing some good ideas to be “aborted” or “still born”. *That conflict between a desire for mathematical purity and a need for “good programwting habits” could only yield a rather anomalous specimen such as ALGOL-60. On the other hand, PL/I, provided a far superior language, complete with the “generative ideas” of third-generation computers.*
- ▶ What their generation lacked most of all, in their opinion, was *the demand for “system elegance”, which is independent of, without necessarily excluding, mathematical “elegance”.*
- ▶ So their caveat to make sure that the modeled object does not become *dominated by the model itself.*

# What is in it for us?



- ▶ This debate, half a century old, has continued relevance for us because of the continued dissonance between theoretical computer science and practical computer and software engineering.
- ▶ In design, development and usage of programming languages it is most influential.
- ▶ Most commonly used programming languages come to most programmers through habits of programming observed and replicated from practitioners.
- ▶ However, courses designed around particular programming languages focus neither on habits nor on usage: they typically focus on syntax—like the third language courses in highschool.

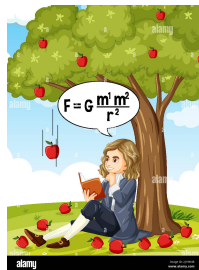


- ▶ The foregoing is most likely to be interpreted as an argument against paradigmatic discipline in programming and in software design.
- ▶ Exactly the opposite is true. The syncretic inference we distilled from the debate (and continued dissonance) between theory and practice in computer science and engineering is as follows.
- ▶ We think that programming paradigms are about **discipline and habits** that help us design and develop software **systems efficiently without compromising on conceptual integrity**.
- ▶ We agree to Dijkstra's "points" (Slide 26) about discipline (that we expressed in our words). But following Brooks [1982], discipline for us is **conceptual integrity** at **human level**, not the mathematical purity that Dijkstra sought.

# Empirical, not Analytical



Thus, we see computer science and engineering, programming paradigms, programming languages, and the conceptual integrity and discipline required of programmers and computer scientists and engineers as *empirical*, like physics, and not *analytical*, like mathematics.



It is not that apples fall *because* Newton defined gravity that way; rather, Newton defined gravity that way *such that* it can *consistently explain* apples falling.

Moreover, Newton's Law of Gravitation does not preclude rockets escaping the earth's gravitational pull. In fact it predicts such a possibility and offers exact calculations to make it happen.

We should look at programming language paradigms and syntax in an analogous manner: they are enablers and provide a scaffolding for (not fetters against) building software systems.



- \* Procedural :
  - ▶ Historically programmers identified repeatedly useful pieces of code and bundled them as “macros” with meaningful code-names (“mnemonics”) to be shared and quickly placed wherever applicable. Gradually, these developed hair and limbs: parameters and adaptive input-output relationships.
- \* Procedural, modular, structured :
  - ▶ Still, this was strictly *procedural* or imperative. But moving beyond mnemonics and formula translation (ForTran), *structured* programming languages arose and facilitated *top-down* design.
- \* Procedural to OO :
  - ▶ Proliferation of adaptive structured programs using common operations repetitively revealed the underlying unity of types and operations, leading to Object-oriented programming



- \* Functional :
  - ▶ An alternative view of operations as transformations led to the functional paradigm.
- \* Logic :
  - ▶ Another alternative model of computation, that of relations (predicates) rather than functions, led to logic programming.

# Four Model Types, Four Paradigms



The (ALU) von Neumann  
Machine, Flow-charts,  
Algorithms, Digital Logic,  
Relay Logic, Ladder  
Diagrams

Imperative Programming

Simulation of evolving  
real-life systems of  
interacting entities

Object-oriented  
Programming

Recursive Function  
Theory, Lambda Calculus

Functional Programming

State Transformations,  
Theorem Proving as  
computation; Relational  
Databases

Logic Programming

# Section



## Preliminaries

- Languages and Abstraction Levels
- Abstraction

## Programming Language Design

- The Clash of Purists and Linguists
- Lessons for Us

## Imperative Paradigm

- Structured Programming
- Principles of Imperative Languages
- Data Types

## Procedural to OO

## Object-Orientation

## Procedure Activation

## OOP in Java

## Generics

## References





- ▶ This set of slides is based on my notes of 2019 and the reference book Sethi [1996, Chapter 3].
- ▶ The main purposes are
  1. to characterize structured programming,
  2. to explain modular programming and its various avatars,
  3. and to expose the main ideology of the C language (to be contrasted with other forms of procedural programming).

... and of course to augment the textbook material (which, in my opinion, is deficient in some aspects).



Programs are made up of global functions and objects. The objects as components of a program can be static and optionally “global” in scope<sup>1</sup>.



Programs are made up of global functions and objects. The objects as components of a program can be static and optionally “global” in scope<sup>1</sup>.

- ▶ Programs are static, computations dynamic.

Data types, sizes, layouts, modules, global functions



Programs are made up of global functions and objects. The objects as components of a program can be static and optionally “global” in scope<sup>1</sup>.

- ▶ Programs are static, computations dynamic.

Data types, sizes, layouts, modules, global functions

- ▶ Control Flow is syntax-directed.

Functions, blocks, loops, conditionals, switch-cases

`break`, `continue`, `goto`



A lucid statement of the philosophy of modular programming can be found in a 1970 textbook on the design of system programs by Gouthier and Pont [1, ¶10.23], which we quote below:<sup>1</sup>

A well-defined segmentation of the project effort ensures system modularity. Each task forms a separate, distinct program module. At implementation time each module and its inputs and outputs are well-defined, there is no confusion in the intended interface with other system modules. At checkout time the integrity of the module is tested independently; there are few scheduling problems in synchronizing the completion of several tasks before checkout can begin. Finally, the system is maintained in modular fashion; system errors and deficiencies can be traced to specific system modules, thus limiting the scope of detailed error searching.



## Compiling:

```
$ gcc structure.c main.c
/tmp/ccMawn27.o: In function `f':
main.c:(.text+0x0): multiple definition of `f'
/tmp/ccIbYZzX.o:structure.c:(.text+0x0): first defined here
collect2: error: ld returned 1 exit status
$ gcc main.c structure.c
/tmp/ccpcS9sT.o: In function `f':
structure.c:(.text+0x0): multiple definition of `f'
/tmp/cc5oXUWo.o:main.c:(.text+0x0): first defined here
collect2: error: ld returned 1 exit status
$ gcc -c structure.c main.c
$ gcc -c main.c structure.c
$ gcc structure.c newmain.c
$ gcc newmain.c structure.c
$ █
```



## Lessons:

1. Each compilation unit is independent in terms of type names, variable names and function names. These names (and structures, for types) have to be supplied to each compilation unit afresh.
2. It is possible to create inconsistencies in data structures using this independence.
3. We can make use of this flexibility to create data structures with due information hiding for system security.
4. But it can lead to logical bugs as well. To avoid that, *header* (\*.h) files are used as *common* includes in various *source* (\*.c) files.

Among the course materials are more programs that exemplify these points and explain them rigorously.



Syntax-directed flow: easy to state. Does this mean the following is syntactically incorrect?

```
#include<stdio.h>
int main(void) {
    goto L1;
    int i;
L2: printf("Input (0 to end) : ");
    scanf("%d",&i);
L1: printf("%d\n",i);
    if(i) goto L2;
    return 0;
}
```

Figure: Control Flow: Example 1

# Control Flow II



```
$ cat controlflow1.c ; gcc controlflow1.c ; ./a.out
#include<stdio.h>
int main(void) {
    goto L1;
    int i;
    L2: printf("Input (0 to end) : ");
        scanf("%d",&i);
    L1: printf("%d\n",i);
    if(i) goto L2;
    return 0;
}
32766
Input (0 to end) : 100
100
Input (0 to end) : 511
511
Input (0 to end) : 0
0
$
```



In effect, it means:

- ▶ *Not* lexical : thus, we cannot assume that the flow will be a straight line like  
→→↓  
→→
- ▶ The *structure* (i.e., syntax, interrelationships and correspondences, with some lexical flow as well, all together) decides the flow, including branches, back-and-forth jumps, etc.
- ▶ Also that basic building blocks, like functions, control blocks (if-else, while, for, switch-case, etc.) are always structured with *single entry, one or more exits*.
- ▶ As Parnas [1972] explains, this means a formal *flow chart* models the program adequately.



In the example 1, we can see that the control flow – made topsy-turvy by the `gotos` – does not affect the *legitimacy* of variable and label names.

## What is Scope?

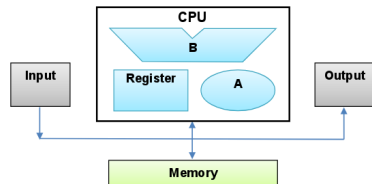
By legitimacy of names, we mean whether a program piece can refer to a name meaningfully and consistently according to the symbolic, algorithmic scheme that the programmer intends. In strict compiler parlance, scope means the contextual part, the textual extent (the part of the source program) in which a particular name can be used and refers to an object unique to that context.

# The Theoretical Basis of Structured Programming



The B ohm and Jacopini [1966] provides the theoretical basis of structured programming. It states that three ways of combining programs—sequencing, selection, and iteration—are sufficient to express any computable function. This observation did not originate with the structured programming movement; these structures are sufficient to describe the instruction cycle of a central processing unit, as well as the operation of a Turing machine. Therefore, a processor is always executing a "structured program" in this sense, even if the instructions it reads from memory are not part of a structured program.

The CPU mentioned above is our von Neumann machine.





- ▶ The von-Neumann machine computes through unit-duration steps specified in unit-sized instructions.
- ▶ Each unit operation is either data or control transfer, or an ALU op.
- ▶ Each such unit operation is an **imperative** operation.
- ▶ These *elementary* unit operations are grouped into *composite* units.
- ▶ Recursively, the composite units of one order can be composed together to make higher order composites.
- ▶ Computation proceeds by executing the bottom level (i.e, elementary) operations in a predetermined *sequence*, possibly with repetition.
- ▶ This sequencing is directly expressed unambiguously in the program units at each level.



- ▶ The design development of these units proceeds in the opposite direction.
- ▶ The highest level composites are expressed first.
- ▶ Each higher level is expressed before its elaboration at the lower level.
- ▶ The bottom-most level is, of course, the *elementary* operations (data and control transfer and ALU operations).
- ▶ However, the compositions of units at every higher level also need to be expressed in terms of the same elementary *operations*. The units are composite at higher levels, but the operations are not.
- ▶ For example, a “`printf(format,...)`” call can be read as:
  - Transfer the string representation of each of the actual parameters in place of the ellipsis in the places indicated by the respective format flags in the format string ...
  - and transfer the resulting string to an stdout buffer.



- ▶ Thus a `printf` call just entails data transfer (and expression evaluation, if any parameters need it).
- ▶ A “`for(E1; E2; E3) { .S. }`” loop is equivalent to “`E1; L1: if(E2) { .S. E3; goto L1 }`”
- ▶ E1 is evaluated once, followed by E2; depending on E2’s value, either the statements .S. are executed followed by evaluation of E3 and looping back to “`if(E2) .S. E3`”, or else the loop ends.
- ▶ This clearly indicates that the *control flow* (*sequencing*—flow—of *imperative*—control—operations) depends on the **Syntax**—*structural relationships* between various components.
- ▶ The last idea is expressed as “Syntax-Directed Control Flow”.



- ▶ Contrast Syntax-Directed Control Flow with *lexical* (textual) flow: someone not familiar with the C/C++/Java for loop syntax would interpret the foregoing as execution of E1, E2, E3, .S. in this sequence repeatedly until something related to E1, E2, E3 together happens.



The following is true of all programming languages (procedural, OO, functional, logic):

- ▶ Each of the loops and blocks (`if`, `while`, `for`, `switch`) has a single point at which control enters it. We can say it is “hooked at the top”.
- ▶ But each can have multiple, unbounded (but finite) exits. Not just `gotos`, but we have `break` too. Of course, `continue` is just like `goto` with a label at the single entry point.
- ▶ Similarly, functions (as also methods in all OO languages) have only single entry points (calls) and multiple exits (returns).



1. Implement recursion on the `main` function in *any* programming language you are comfortable with among C/C++/Java.
2. Using this, print the command line arguments to a program in the reverse order.
3. Write a set of functions (procedures) that call one another in such recursive cycles that one cannot predict just by reading the program whether recursion will end and whether a certain function will be called for a given input.
4. Provide a body for the `printf` function and use that function instead of the `std libc` function with the same name in your C program.
5. Implement a type that can only be used by calling the procedures you provide to use it, and even its contents cannot be accessed or displayed by the user programmer.

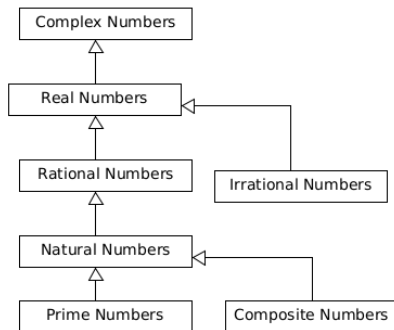


6. Using only `math.h` or `numpy` function calls and arithmetic expressions but no assignment statement, generate and print an input integer.

# Types as just a Set of Values



A “class hierarchy view” of some of the well-known sets of numbers:



- ▶ Can we say that “natural numbers have all the properties of real numbers, and also some additional special properties”?
- ▶ For a method call in Java, can we pass a complex number as an actual parameter in a place wherein a natural number is expected as a formal parameter?

# Section



## Preliminaries

- Languages and Abstraction Levels
- Abstraction

## Programming Language Design

- The Clash of Purists and Linguists
- Lessons for Us

## Imperative Paradigm

- Structured Programming
- Principles of Imperative Languages
- Data Types

## Procedural to OO

### Object-Orientation

### Procedure Activation

### OOD in Java

### Generics

### References



# Separation of the *What* and the *How*



- ▶ We want to name what is programmable by meaningful names that help us recall what they stand for.
- ▶ Naming a list such that the name tells us that it is a list and it remains sorted is easy: name it `SORTEDLIST`
- ▶ It doesn't matter whether the `SORTEDLIST` is implemented as an array or any other *storage* structure, or *how* it is kept sorted, by `INSERTIONSORT` or any other algorithm.
- ▶ All we need is a *data* structure that offers *methods* to put elements in and remove from, and give them out in a sorted manner (i.e., a method to get the “ $n^{\text{th}}$  element”, the element that has rank  $n$  in the sorted order.
- ▶ Naming a procedure that sorts by recursively partitioning the input list around a pivot until the partitions are smaller than 3 elements—the one we know as `QUICKSORT`—is not easy. [But then, a procedure's *process* (i.e. the *how* part), once programmed, is not further “programmable” anyway.]

# Why Encapsulation?



- ▶ If a procedure's identity is its algorithm that doesn't change, and a data structure's identity is not dependent on the algorithms that maintain its characteristics, then where is the incentive to encapsulate data and operations together?
- ▶ "The benefits expected of modular programming are: (1) managerial—development time should be shortened because separate groups would work on each module with little need for communication; (2) product flexibility—it should be possible to make drastic changes to one module without a need to change others; (3) comprehensibility—it should be possible to study the system one module at a time. The whole system can therefore be better designed because it is better understood." Parnas [1972]
- ▶ In this, a module indicates *responsibility delegation*, not just *division of labour*.
- ▶ Modularization involves *system* (top) level design decision that must be made *before* the detailing (bottom-level) work on modules begins.



Parnas [1972] presents two modularizations for a small text processor (called an “example system” in it). One modularization is of the “conventional type” in the sense meant by all proponents of modular programming: *The system is divided into a number of modules with well-defined interfaces; each one is small enough and simple enough to be thoroughly understood and well programmed.* The other modularization or decomposition was made using “information hiding” as a criterion. The modules no longer correspond to steps in the processing. Every module in the second decomposition is characterized by its knowledge of a design decision which it hides from all others. Its interface or definition was chosen to reveal as little as possible about its inner workings. We can call this (rudimentary) OO decomposition.



- ▶ BUBBLESORT gets its name from its process: “lighter elements *bubble* up”.
- ▶ Similarly MERGESORT, INSERTIONSORT, etc. get their names from their process or the sequence of transformations in sorting.
- ▶ What about QUICKSORT? Can you make a name that captures its process like those names of the other sorting algorithms?

**Exercise:** Design (at the top level) a dictionary that guarantees  $O(\log n)$  operations for insertion, removal, key-based retrieval, “ $k^{\text{th}}$  rank element” retrieval; also guarantees  $O(n)$  listing of elements by sorted order ( $n$  is the size). It must also guarantee *stability*: if two elements are at the same rank, they should appear in their relative FIFO sequence in a sorted retrieval, and the element inserted earlier should be retrieved for a single element key-based search.

# Section



## Preliminaries

- Languages and Abstraction Levels
- Abstraction

## Programming Language Design

- The Clash of Purists and Linguists
- Lessons for Us

## Imperative Paradigm

- Structured Programming
- Principles of Imperative Languages
- Data Types

## Procedural to OO

## Object-Orientation

## Procedure Activation

## OOP in Java

## Generics

## References



# More than just a division of labour



- ▶ So far we have seen separation of the *what* (specifications of data types and operations) and the *how* (storage layouts, sizes, procedures) *within* the procedural paradigm.
- ▶ Sethi [Sethi, 1996, p.197, 6.8] says that considerations of efficiency and implementation led to the (criticised) design that private members of a class appear in a class declaration.
- ▶ Following Parnas Parnas [1972], we see that object-orientation involved “information *hiding*”—meaning, not just separation for modularity but also *specialisation and independence*.
- ▶ Thus, Booch [Booch, 2007, pp.51–52] defines encapsulation as “the process of compartmentalizing the elements of an abstraction that constitute its structure and behavior; encapsulation serves to separate the contractual interface of an abstraction and its implementation.” In this sense, encapsulation is the concrete way to implement the abstract idea of independence.



## Imperative

Procedural programming involves giving a complete step-by-step break-up of the execution of a full solution to a computational problem. The break-up may be modularised, but ultimately it is compiled and assembled into a sequential program.

## Emergent

In OO programming an information process is regarded as a system evolving through transformations of its state. The essence (substance) of the information process is organized as the system components, called objects. A measurable property of the essence is a computational property of some objects. Transformations of state are regarded as actions by objects.

# Simulation means OO Programming



In OO programming an information process is regarded as a system evolving through transformations of its state. The essence (substance) of the information process is organized as the system components, called objects. A measurable property of the essence is a computational property of some objects. Transformations of state are regarded as actions by objects. This is paraphrasing Nygaard Nygaard [1986], who pioneered—with Ole-Johan Dahl—Simula, the first ever OO programming language, that started off as a simulation language. He describes it thus: “Simula I, designed in 1962-64 and available in 1965, was immediately used outside simulation, as a general programming language: The computing process could by Simula be organized as system of interacting program execution components, and this approach proved to be fruitful in a wide range of application areas. This, and some new ideas (e.g. subclasses and virtuals) resulted in the current Simula language, Simula 67, which is a general programming language, also used as a simulation language.”

# OO on top of Procedural is Fraught



We see a limitation of C++: Not only that a class declaration (or type definition) must be fully available (including private members) to each module that uses that class, the procedural details of its storage structure must be fully laid out; moreover, keeping the integrity and consistency of that structure is left to each module—unless special care is taken by the class module that implements the class. There is no direct and enforceable support in the syntax of C++ for this.

The next slides show one example of this shortcoming (and how it can create timebombs in code written by benign but less careful programmers).

# A “Partially Hidden” Class



Suppose there are two modules that use two declarations for the same named class:

The Class Module

```
class FullClass {  
    private :  
        int objectId, *data, size;  
    public :  
        FullClass(void);  
        FullClass(int);  
        FullClass(int, int*);  
        FullClass(const FullClass&);  
        void tell(void);  
        void setNthElement(int,int);  
        ~FullClass(void);  
};
```

|| The User Module

```
class FullClass {  
    public :  
        FullClass(void);  
        FullClass(int);  
        FullClass(int, int*);  
        FullClass(const FullClass&);  
        void tell(void);  
        void setNthElement(int,int);  
        ~FullClass(void);  
};
```

All the member functions (methods) of the class are in the class module. The user module is developed *independently*, without regard to the implementation of the class module. (That's the purpose in all the information hiding, isn't it?)



To understand this, we need to understand three important points; one is a general principle of programming languages, and the others are features of modular (structured) imperative languages.

1. *Procedure activation records*: Calls to methods of a class are no different from calls to global functions, except that “this” object is a default and implicit (first) parameter. Parameter passing and return mechanisms are exactly the same. These mechanisms operate via procedure activation records on a stack, with stack frames separated for each call (regardless of which function it is). We will see this in more detail and depth in the next section. But for the current purpose, it suffices to note that the “this” object must be passed as a storage structure, that means, its data members are to be passed onto the stack. This is the semantics of call-by-value situations in all programming languages.



2. Each object (of a primitive type like `int`, `float` etc. or a composite, like an array or a struct or a class instance) that is to be passed onto the stack, or allocated space automatically, uses the size and layout information of that object available in the scope of the function call. That means, the type declaration available in the *scope* of the module and/or the block in which the call is enclosed. For imperative languages, this is a *static*, compile-time piece of information, and it is interpreted in the procedure activation time (call) translation.
3. For structured (modular procedural) languages like C (and thereby C++ that has C as a subset, with the same program structure), each module must be independent in static typing. That means, each module has its own type information, and that information is processed and converted into some quantitative machine level data, losing the human level



symbolic names. The linker does not see the type names, the names of variables, parameters and data members, etc. except for global variables, static variables, and global function names. The names of the class methods (member functions) are also *mangled* with the class name incorporated in the new names. Thus, if two modules access different data declarations in class declarations, the compiler does not see the incongruity and the linker has no access to (and no business with) that information at all after compilation.

Thus, two modules using incongruent, differing type declarations is usually passed over by the compiler, and the linker does not see the incongruity unless it appears in function prototypes (or method signatures). Hence the programs will compile, link together, and run partially. However, there will be semantic errors leading to runtime exceptions and program crashes. The programs uploaded on the course page can be used (and debugged) to understand this in more depth.



Simply put, the foregoing means objects are not the principal components of a C/C++ program. This is true of every imperative language that has the program composed of modules and global functions, and class definitions, types are also implemented as such. There is encapsulation but modules and functions are indivisible and independent, whereas objects and types are not. The rigorous definition of OO programming by Nygaard Nygaard [1986], that, object-oriented programming regards an information process as a system evolving through transformations of its state that is composed of the states of independently interacting evolving objects, is not truly realised in such a programming language. This is the most important difference between C++ and similar languages (e.g. the modern Fortran with added object orientation) on the one hand and Java, Simula, SmallTalk, and functional-OO languages like Python etc. on the other hand.



Objective-C and its more advanced derivatives differ from C++ in this important aspect: they treat methods of a class in a different way from ordinary, global functions.



Going from procedural to OO, the most important aspects are

1. Scope of type expressions
2. Object independence
3. Treatment of class member functions

Java has no global functions (i.e., no functions outside classes), and type linkages are all global (except for specifically local types within classes). Type information necessary for bindings is present throughout the program's execution. Thus, Java is “purely” OO. In the next section, we see the aspects of type and function parameter bindings common to all paradigms and languages.

# Section



## Preliminaries

- Languages and Abstraction Levels
- Abstraction

## Programming Language Design

- The Clash of Purists and Linguists
- Lessons for Us

## Imperative Paradigm

- Structured Programming
- Principles of Imperative Languages
- Data Types

## Procedural to OO

## Object-Orientation

## Procedure Activation

## OOP in Java

## Generics

## References





A typical division of labour in a compiled program's production and application process is:



Compilation from source to assembly or byte-code or machine-code



Linking various modules and static libraries together



Loading in the runtime environment: e.g. the main memory of a CPU or a virtual machine



Execution

# The Lifetime of a Program



Development Design, write, test, pickle.

# The Lifetime of a Program



**Development** Design, write, test, pickle.

**Loading** May include linking and spawning other programs.

# The Lifetime of a Program



**Development** Design, write, test, pickle.

**Loading** May include linking and spawning other programs.

**Execution** May include human-interactive interpretation and interaction with other programs running on the same host or other hosts.

# The Lifetime of a Program



**Development** Design, write, test, pickle.

**Loading** May include linking and spawning other programs.

**Execution** May include human-interactive interpretation and interaction with other programs running on the same host or other hosts.

## Finer Distinctions

# The Lifetime of a Program



**Development** Design, write, test, pickle.

**Loading** May include linking and spawning other programs.

**Execution** May include human-interactive interpretation and interaction with other programs running on the same host or other hosts.

## Finer Distinctions

**Compile Time** Interpreting and translating the intention of the programmer as expressed in the text.

# The Lifetime of a Program



**Development** Design, write, test, pickle.

**Loading** May include linking and spawning other programs.

**Execution** May include human-interactive interpretation and interaction with other programs running on the same host or other hosts.

## Finer Distinctions

**Compile Time** Interpreting and translating the intention of the programmer as expressed in the text.

**Linking Time** Combining and composing the translated programmer's text with the "standard environment and support".

# The Lifetime of a Program



**Development** Design, write, test, pickle.

**Loading** May include linking and spawning other programs.

**Execution** May include human-interactive interpretation and interaction with other programs running on the same host or other hosts.

## Finer Distinctions

**Compile Time** Interpreting and translating the intention of the programmer as expressed in the text.

**Linking Time** Combining and composing the translated programmer's text with the "standard environment and support".

**Loading Time** Preparing execution, including the dynamic part of the linking.

# The Lifetime of a Program



**Development** Design, write, test, pickle.

**Loading** May include linking and spawning other programs.

**Execution** May include human-interactive interpretation and interaction with other programs running on the same host or other hosts.

## Finer Distinctions

**Compile Time** Interpreting and translating the intention of the programmer as expressed in the text.

**Linking Time** Combining and composing the translated programmer's text with the "standard environment and support".

**Loading Time** Preparing execution, including the dynamic part of the linking.

**Procedure Activation Time** Each time a piece of code that can be executed recurrently and composed as a unit, is to be executed from a specific entry point.

# The Lifetime of a Program



**Development** Design, write, test, pickle.

**Loading** May include linking and spawning other programs.

**Execution** May include human-interactive interpretation and interaction with other programs running on the same host or other hosts.

## Finer Distinctions

**Compile Time** Interpreting and translating the intention of the programmer as expressed in the text.

**Linking Time** Combining and composing the translated programmer's text with the "standard environment and support".

**Loading Time** Preparing execution, including the dynamic part of the linking.

**Procedure Activation Time** Each time a piece of code that can be executed recurrently and composed as a unit, is to be executed from a specific entry point.

**Run Time** Each atomic *imperative* piece of code at the machine level.

# What is in name-calling?



- ▶ The first abstractions in programming were macros. The expressive power of functions with parameters and return values was realised much later. Till now, the power of recursion (and composition) of functions is scarcely understood and appreciated by programmers involved in software engineering on scale.
- ▶ There are functions that compute the value of a mapping of parameters onto a domain of values. And there are pure procedures that only encapsulate some long winding process into a neatly named subroutine. Instead of distinguishing, let us call all “callable”s as subroutines. And a call as “Procedure Activation”, following Sethi [1996].

The treatment here is mainly following Sethi [1996, Chapter 5]. However, we have tried to supplement the material with insights from Scott [2015, Chapter 3, Chapter 8], especially 3.8.



There are six main features (including components) to a subroutine:

1. The name and the signature (or prototype)
2. Parameter passing and access (preparation to call and subroutine preamble)
3. Allocation and access to local (including temporary) objects, access to non-local objects
4. The imperative action procedure (translation of the source code body of the subroutine)
5. Preparation to return (return values, winding up operations such as C++ destructor calls for objects created locally at procedure activation, etc.)
6. Restoration of the program state at return



We demonstrate the six elements of subroutines listed above in a recursive function. Observe the source code and its translation into assembly language with debugging info.

```
int gcd(int parama , int paramb) {  
    static int xstatic = 0;  
  
    xstatic = parama-paramb;  
    if(parama>0 && paramb>0) {  
        if(xstatic<0)  
            return gcd(parama,-xstatic);  
        else if(xstatic>0)  
            return gcd(paramb,xstatic);  
        else return parama;  
    }  
    else return -1;  
}
```



The output of `gcc -g -S euclid.c`. The `.loc` assembler directives correspond to the line numbers in the source code. Before the first such directive is the preamble. Call preparation (parameter passing etc.) can be seen between the corresponding source lines.

```
.file    "euclid.c"
.text
.Ltext0:
.file 0  "/home/rs-joshi/sangrah/
        Courses/PoPL/DemoProgs/
        ProcedureActivation" "euclid.c"
.globl  gcd
.type   gcd, @function

gcd:
.LFB0:
.file 1  "euclid.c"
```



```
.loc 1 1 33
.cfi_startproc
endbr64
pushq    %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq     %rsp, %rbp
.cfi_def_cfa_register 6
subq     $16, %rsp
movl     %edi, -4(%rbp)
movl     %esi, -8(%rbp)
.loc 1 4 18
movl     -4(%rbp), %eax
subl     -8(%rbp), %eax
.loc 1 4 10
movl     %eax, xstatic.0(%rip)
.loc 1 5 4
```



```
    cmpl    $0, -4(%rbp)
    jle     .L2
    .loc 1 5 14 discriminator 1
    cmpl    $0, -8(%rbp)
    jle     .L2
    .loc 1 6 14
    movl    xstatic.0(%rip), %eax
    .loc 1 6 6
    testl   %eax, %eax
    jns     .L3
    .loc 1 6 25 discriminator 1
    movl    xstatic.0(%rip), %eax
    negl    %eax
    movl    %eax, %edx
    movl    -4(%rbp), %eax
    movl    %edx, %esi
    movl    %eax, %edi
```



```
    call    gcd
    jmp     .L4

.L3:
    .loc 1 7 19
    movl    xstatic.0(%rip), %eax
    .loc 1 7 11
    testl   %eax, %eax
    jle     .L5
    .loc 1 7 30 discriminator 1
    movl    xstatic.0(%rip), %edx
    movl    -8(%rbp), %eax
    movl    %edx, %esi
    movl    %eax, %edi
    call    gcd
    jmp     .L4

.L5:
    .loc 1 8 16
```



```
    movl    -4(%rbp), %eax
    jmp     .L4

.L2:

    .loc 1 10 14
    movl    $-1, %eax

.L4:

    .loc 1 11 1
    leave
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc

.LFE0:

    .size    gcd, .-gcd
    .local   xstatic.0
    .comm    xstatic.0,4,4

.Letext0:
```



```
.section                .debug_info ,"" ,
                        @progbits
.Ldebug_info0:
    .long    0x85
    .value   0x5
    .byte    0x1
    .byte    0x8
    .long    .Ldebug_abbrev0
    .uleb128 0x2
    .long    .LASF4
    .byte    0x1d
    .long    .LASF0
    .long    .LASF1
    .quad    .Ltext0
    .quad    .Letext0 - .Ltext0
    .long    .Ldebug_line0
    .uleb128 0x3
```



```
.string "gcd"
.byte    0x1
.byte    0x1
.byte    0x5
.long    0x81
.quad    .LFB0
.quad    .LFE0-.LFB0
.uleb128 0x1
.byte    0x9c
.long    0x81
.uleb128 0x1
.long    .LASF2
.byte    0xd
.long    0x81
.uleb128 0x2
.byte    0x91
.sleb128 -20
```



```
.uleb128 0x1
.long    .LASF3
.byte    0x19
.long    0x81
.uleb128 0x2
.byte    0x91
.sleb128 -24
.uleb128 0x4
.long    .LASF5
.byte    0x1
.byte    0x2
.byte    0xd
.long    0x81
.uleb128 0x9
.byte    0x3
.quad    xstatic.0
.byte    0
```



```
.uleb128 0x5
.byte 0x4
.byte 0x5
.string "int"
.byte 0
.section .debug_abbrev,"",
        @progbits
.Ldebug_abbrev0:
.uleb128 0x1
.uleb128 0x5
.byte 0
.uleb128 0x3
.uleb128 0xe
.uleb128 0x3a
.uleb128 0x21
.sleb128 1
.uleb128 0x3b
```



```
.uleb128 0x21
.sleb128 1
.uleb128 0x39
.uleb128 0xb
.uleb128 0x49
.uleb128 0x13
.uleb128 0x2
.uleb128 0x18
.byte 0
.byte 0
.uleb128 0x2
.uleb128 0x11
.byte 0x1
.uleb128 0x25
.uleb128 0xe
.uleb128 0x13
.uleb128 0xb
```



```
.uleb128 0x3
.uleb128 0x1f
.uleb128 0x1b
.uleb128 0x1f
.uleb128 0x11
.uleb128 0x1
.uleb128 0x12
.uleb128 0x7
.uleb128 0x10
.uleb128 0x17
.byte 0
.byte 0
.uleb128 0x3
.uleb128 0x2e
.byte 0x1
.uleb128 0x3f
.uleb128 0x19
```



```
.uleb128 0x3
.uleb128 0x8
.uleb128 0x3a
.uleb128 0xb
.uleb128 0x3b
.uleb128 0xb
.uleb128 0x39
.uleb128 0xb
.uleb128 0x27
.uleb128 0x19
.uleb128 0x49
.uleb128 0x13
.uleb128 0x11
.uleb128 0x1
.uleb128 0x12
.uleb128 0x7
.uleb128 0x40
```



```
.uleb128 0x18
.uleb128 0x7c
.uleb128 0x19
.uleb128 0x1
.uleb128 0x13
.byte    0
.byte    0
.uleb128 0x4
.uleb128 0x34
.byte    0
.uleb128 0x3
.uleb128 0xe
.uleb128 0x3a
.uleb128 0xb
.uleb128 0x3b
.uleb128 0xb
.uleb128 0x39
```



```
.uleb128 0xb
.uleb128 0x49
.uleb128 0x13
.uleb128 0x2
.uleb128 0x18
.byte 0
.byte 0
.uleb128 0x5
.uleb128 0x24
.byte 0
.uleb128 0xb
.uleb128 0xb
.uleb128 0x3e
.uleb128 0xb
.uleb128 0x3
.uleb128 0x8
.byte 0
```



```
.byte    0
.byte    0
.section          .debug_aranges,"",
    @progbits
.long    0x2c
.value   0x2
.long    .Ldebug_info0
.byte    0x8
.byte    0
.value   0
.value   0
.quad    .Ltext0
.quad    .Ltext0-.Ltext0
.quad    0
.quad    0
.section          .debug_line,"",
    @progbits
```



```
.Ldebug_line0:
    .section          .debug_str,"MS",
        @progbits,1

.LASF4:
    .string "GNU C17 11.4.0 -mtune=generic
        -march=x86-64 -g -fasynchronous-
        unwind-tables -fstack-protector-
        strong -fstack-clash-protection -
        fcf-protection"

.LASF3:
    .string "paramb"

.LASF2:
    .string "parama"

.LASF5:
    .string "xstatic"
    .section          .debug_line_str,"MS",
        @progbits,1
```



.LASF0:

.string "euclid.c"

.LASF1:

.string "/home/rs-joshi/sangrah/  
Courses/PoPL/DemoProgs/  
ProcedureActivation"

.ident "GCC: (Ubuntu 11.4.0-1ubuntu1  
~22.04) 11.4.0"

.section .note.GNU-stack,"",  
@progbits

.section .note.gnu.property,"a"

.align 8

.long 1f - 0f

.long 4f - 1f

.long 5

0:

.string "GNU"



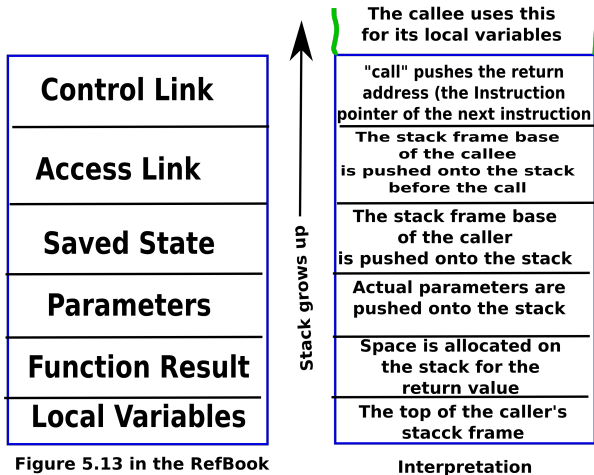
```
1:      .align 8
        .long  0xc0000002
        .long  3f - 2f

2:      .long  0x3

3:      .align 8

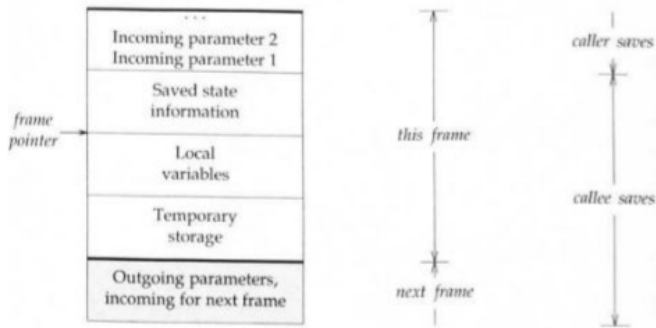
4:
```

# The Stack Frame



Ref: Sethi [1996, page 146, Figure 5.13]

# C and C++ Procedure Activations



Sethi [1996, Figure 5.19]

We explain the corresponding assembly instructions next.



- ▶ Incoming parameters: Pushed by the caller onto the stack before issuing the call instruction.
- ▶ The call instruction saves the state (return address; if re-entrant code then the PSW, etc.)
- ▶ The (callee) frame pointer is now fixed at this point.

```
.cfi_startproc
endbr64
pushq    %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq     %rsp, %rbp
.cfi_def_cfa_register 6
subq     $16, %rsp
movl     %edi, -4(%rbp)
movl     %esi, -8(%rbp)
```



Local variables and temporary storage are calculated before doing these offset adjustments. Before this, of course, the BP (the frame pointer of the caller's frame) is saved in the “saved state information” block. This is the access link.

- ▶ Outgoing parameters for the call on line 6 of gcd.c:

```
movl    xstatic.0(%rip), %eax
negl    %eax
movl    %eax, %edx
movl    -4(%rbp), %eax
movl    %edx, %esi
movl    %eax, %edi
call    gcd
jmp     .L4
```

- ▶ After all recursion ends, the winding up part:



```
.L4:  
    leave  
    .cfi_def_cfa 7, 8  
    ret  
    .cfi_endproc
```

- Note the “recursive returns”—after each recursive call, the return value is assumed to be there in the expected location (in this instance, the accumulator register) and control is transferred to the `leave` instruction above. Only for lines 8 (else return parama) and 10 (else return -1) there is explicit result movement:

# Call and Subroutine IV



```
.L5:
    .loc 1 8 16
    movl    -4(%rbp), %eax
    jmp     .L4
.L2:
    .loc 1 10 14
    movl    $-1, %eax
.L4:
```



Along with the gcd program, we experimented with many C and C++ programs, often debugging them line by line and examining the stack frames. Here are the important questions we sought to solve by this:

- ▶ Which names are retained in the symbol table for the linker?
- ▶ Which names are converted into offsets? How?
- ▶ How are method calls in C++ handled? In the same way as procedure activation for global functions, or differently?
- ▶ How is polymorphism enabled in C++? How is the class name preserved, how are method signatures uniquely identified so that membership, class hierarchy, overloading, overriding, virtual methods (abstraction), reflection etc. are handled?

# Section



## Preliminaries

- Languages and Abstraction Levels
- Abstraction

## Programming Language Design

- The Clash of Purists and Linguists
- Lessons for Us

## Imperative Paradigm

- Structured Programming
- Principles of Imperative Languages
- Data Types

## Procedural to OO

## Object-Orientation

## Procedure Activation

## **OOP in Java**

## Generics

## References



# When to Java?



- ▶ Mainly for interfacing (not just UI, but for development without the nitty-gritty)

# When to Java?



- ▶ Mainly for interfacing (not just UI, but for development without the nitty-gritty)
- ▶ When the focus is the core business logic of the application, not the implementation

# When to Java?



- ▶ Mainly for interfacing (not just UI, but for development without the nitty-gritty)
- ▶ When the focus is the core business logic of the application, not the implementation
- ▶ When the user and system requirements prompt the choices of libraries and class hierarchies

# When to Java?



- ▶ Mainly for interfacing (not just UI, but for development without the nitty-gritty)
- ▶ When the focus is the core business logic of the application, not the implementation
- ▶ When the user and system requirements prompt the choices of libraries and class hierarchies

# When to Java?



- ▶ Mainly for interfacing (not just UI, but for development without the nitty-gritty)
- ▶ When the focus is the core business logic of the application, not the implementation
- ▶ When the user and system requirements prompt the choices of libraries and class hierarchies

Then, wouldn't LLMs replace Java programmers?

# When Not to Java?



## ► The *Banana Monkey Jungle* Problem

# When Not to Java?



- ▶ The *Banana Monkey Jungle* Problem
- ▶ The problem with the “contain and delegate” solution  
<https://cscalfani.medium.com/goodbye-object-oriented-programming-a59cda4c0e53>

# When Not to Java?



- ▶ The *Banana Monkey Jungle* Problem
- ▶ The problem with the “contain and delegate” solution  
<https://cscalfani.medium.com/goodbye-object-oriented-programming-a59cda4c0e53>
- ▶ Is “reusability” such a holy or useful thing in the times of LLMs?



- ▶ The *Banana Monkey Jungle* Problem
- ▶ The problem with the “contain and delegate” solution  
<https://cscalfani.medium.com/goodbye-object-oriented-programming-a59cda4c0e53>
- ▶ Is “reusability” such a holy or useful thing in the times of LLMs?
- ▶ Polymorphism, Contracts and Flexibility: the “tension-triad”



1. Make a 2-class hierarchy, instantiate any one of the two classes, make this instance forget what it was to begin with and instead imagine itself to be of the other class.
2. Make a single instance of a class deviate in a call to a method from all other instances of the class.
3. Replace an instance of a superclass by that of a subclass after requiring the corresponding specialization and waiting for that specialization to be developed and available.
4. Make a generic method adapt to a new specialization.

# Section



## Preliminaries

- Languages and Abstraction Levels
- Abstraction

## Programming Language Design

- The Clash of Purists and Linguists
- Lessons for Us

## Imperative Paradigm

- Structured Programming
- Principles of Imperative Languages
- Data Types

## Procedural to OO

## Object-Orientation

## Procedure Activation

## OOP in Java

## Generics

## References





Modern programming language design lives in tension between two apparently conflicting demands. On the one hand, high-level languages such as Scheme, Prolog, and ML incorporate significant advances in elegance and expressive power. On the other hand, many industrial programmers find these languages too "theoretical" or impractical for everyday use, and too inefficient. As a result, these languages are often used only in academic and research contexts, while the majority of the world's mainline programming is conducted in such languages as C and C++, distinguished instead for their efficiency and adaptability.

This book is about a new approach to programming language design, in which these two demands of elegance and efficiency are viewed as compatible, not conflicting. Our goal is the development of languages that are as clean as the purest theoretical designs, but that make no compromises on performance or control over implementation.

# The MetaObject Protocol II



The way in which we have achieved elegance and efficiency jointly is to base language design on metaobject protocols. Metaobject protocols are interfaces to the language that give users the ability to incrementally modify the language's behavior and implementation, as well as the ability to write programs within the language.

Languages that incorporate metaobject protocols blur the distinction between language designer and language user. Traditionally, designers are expected to produce languages with well-defined, fixed behaviors (or "semantics"). Users are expected to treat these languages as immutable black-box abstractions, and to derive any needed flexibility or power from constructs built on top of them. This sharp division is thought to constitute an appropriate division of labor. Programming language design is viewed as a difficult, highly-specialized art, inappropriate for average users to engage in. It is also often assumed that a language design must be rigid in order to support portable implementations, efficient compilers, and the like.



- ▶ **Class Redefinition:** allows the definition of a class to be changed; the changes are propagated to its subclasses and to extant instances.
- ▶ **Method Redefinition:** replaces the current definition completely by the new definition.
- ▶ **Forward-Referencing of Superclasses:** allows classes to be referenced before they are defined. One class can be defined in terms of another before the second has been defined.
- ▶ **Implicit (inferred) Generic Function Definitions:** allows the definition of a generic function to be inferred from the method definitions.
- ▶ **User Control of Method Combination:** It provides a mechanism for customizing the production of the effective method; in the full form, mechanisms for implementing arbitrary control structures within method combination and for arbitrary processing of method qualifiers.

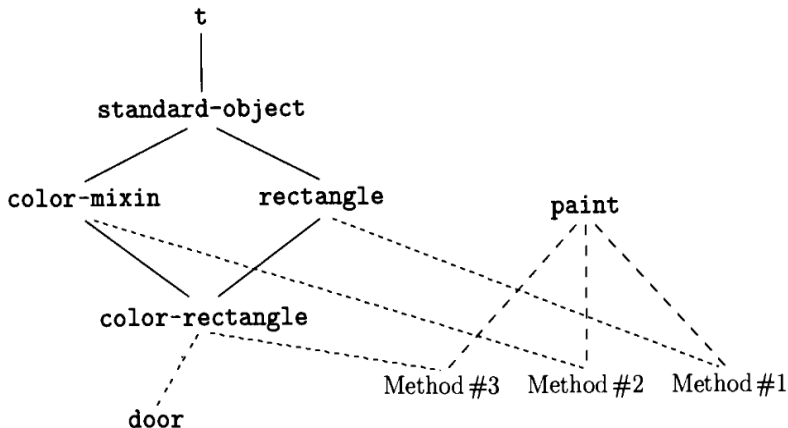


- ▶ **The “eq1” Specializers:** allows methods to be specialized not only to classes, but also to individual objects.
- ▶ **Slots with “:” Class Allocation:** supports slots allocated in each instance of a class and slots which are shared across all of them.
- ▶ **Fully Integrated Types and Classes:** closely integrates Common Lisp types and CLOS classes. It is possible to define methods specialized to primitive classes (e.g., `symbol`) and structure classes (defined with `defstruct`).



- ▶ In its simplest terms, a CLOS program consists of `defclass`, `defgeneric`, and `defmethod` forms mixed in with other more traditional Common Lisp forms. Executing these forms defines the program's classes, generic functions and methods.
- ▶ Backstage, execution of these forms creates internal representations of the classes, generic functions, and methods, recording the information provided in their definitions. The implementation uses the information stored in the internal representation of a class to create instances of that class and to access their slots. Information stored in the internal representation of a generic function and its methods is used to invoke the generic function.

# An Example: Painting



- Direct-superclass/subclass
- - - Generic-function/Methods
- ..... Specializer/Direct-method
- - - Class-of



- ▶ `(find-class <symbol> &optional <errorp>)` Returns the class metaobject corresponding to the class named by `<symbol>`. If there is no class with this name and `<errorp>` is either missing or non-nil, an error is signaled.
- ▶ If there is no class with this name and `<errorp>` is nil, then nil is returned.
- ▶ `(class-of <object>)` Returns the class metaobject corresponding to the class of `<object>`.



## DESIGN & IMPLEMENTATION

### 12.2 Reflection

A *reflection* mechanism allows a program to reason about itself. While no widely used language is *fully reflective*, in the sense that it can inspect every aspect of its structure and current state, significant forms of *reflection* appear in several major languages, Prolog among them. Given the functor and arity of a starting goal, the `clause` predicate allows us to find everything related to that goal in the database. Using `clause`, we can in fact create a *metacircular interpreter* (Exercise 12.13)—an implementation of Prolog in itself—much as we could for Lisp using `eval` and `apply` (Section 11.3.5). We can also write evaluators that use nonstandard search orders (e.g., breadth-first or forward chaining; see Exercise 12.14). Other examples of rich *reflection* facilities appear in Java, C#, and the major scripting languages. As we shall see in Section 16.3.1, these allow a program to inspect and reason about its complete type structure. A few languages (e.g., Python) allow a program to inspect its source code as text, but this is not as powerful as the homoiconic inspection of Prolog or Scheme, which allows a program to *reason about* its own code structure directly.



- ▶ Under the rules of standard method combination, applicable before-methods are executed first, *from most specific to least specific*. The *most specific* applicable primary method is executed next, followed by the applicable after-methods, *from least specific to most specific*. The value(s) returned by the call to the generic function are the values(s) returned by the primary method.
- ▶ The function `apply-methods` does this sequencing, calling `apply-method` to invoke individual methods on the arguments.
- ▶ Applying a method is handled in basically the same way a Lisp interpreter handles a call to a normal (non-primitive) function: evaluate the body form in the lexical environment of the `defmethod` as extended with new variable binding resulting from matching the lambda list against the list of actual arguments.



Engineer academicians working on an industrial project of design and development of a metaprogramming language with Europolitan Vodafone AB and Diedata AB say:

## Reflection

There are two types of reflection: introspection and intercession ... The purpose of introspection is to acquire information about the program itself and to use that information within the program.

Intercession goes further. It allows the program to alter its own behavior. Different programming languages have different reflection capabilities. Languages such as Lisp or Smalltalk have both introspection and intercession, while Java is basically introspective only.



- ▶ Naive implementations of the MOP are woefully inefficient. At the very least, any real implementation must handle generic function invocation and slot access more efficiently.
- ▶ The basis for optimizing implementations is the idea: **improve performance by computing the results of critical internal computations once and then saving and reusing that result.** This technique is known as *memoization*.

# Section



## Preliminaries

- Languages and Abstraction Levels
- Abstraction

## Programming Language Design

- The Clash of Purists and Linguists
- Lessons for Us

## Imperative Paradigm

- Structured Programming
- Principles of Imperative Languages
- Data Types

## Procedural to OO

## Object-Orientation

## Procedure Activation

## OOP in Java

## Generics

## References



Corrado Böhm and Giuseppe Jacopini. Flow diagrams, turing machines and languages with only two formation rules.

*Commun. ACM*, 9(5 (May 1966)):366–371, 1966. URL <https://doi.org/10.1145/355592.365646>.

Grady Booch. *Object-Oriented Analysis and Design with Applications*. Addison-Wesley, 2007. ISBN 978-0-201-89551-3.

Jr. Brooks, Frederick P. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley Pub. Co., Chicago, 1982.

Edsger W. Dijkstra. Letters to the editor: go to statement considered harmful. *Commun. ACM*, 11(3 (March 1968)): 147–148, 1968. URL <https://doi.org/10.1145/362929.362947>.

Edsger W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, N.J., USA, 1976.



- Robert J. DuWorks and Steven W. Smoliar. The arrogant programmer: Dijkstra and Wegner considered harmful. *SIGCSE Bull.*, 4(4 (Dec. 1972)):19–21, 1972. URL <https://doi.org/10.1145/382224.382454>.
- David Gries. *The Science of Programming*. Springer Verlag, N.Y., USA, 1981.
- M. Stuart Lynn. Acm forum. *Commun. ACM*, 15(4 (April 1972)): 280, 1972. URL <https://doi.org/10.1145/361284.361302>.
- Kristen Nygaard. Basic concepts in object oriented programming. In *Proceedings of the 1986 SIGPLAN Workshop on Object-Oriented Programming*, OOPWORK '86, page 128–132, New York, NY, USA, 1986. Association for Computing Machinery. ISBN 0897912055. doi: 10.1145/323779.323751. URL <https://doi.org/10.1145/323779.323751>.



- D. L. Parnas. On the criteria to be used in decomposing systems into modules. *Commun. ACM*, 15(12 (Dec. 1972)):1053–1058, 1972. URL <https://doi.org/10.1145/361598.361623>.
- John R. Rice and Edsger W. Dijkstra. Letters to the editor: The go to statement reconsidered. *Commun. ACM*, 11(8 (Aug. 1968)): 538, 1968. URL <https://doi.org/10.1145/363567.363570>.
- Michael L. Scott. *Programming Language Pragmatics*. Morgan-Kaufmann (Elsevier), 4 edition, 2015. ISBN 9780124104099, 9780124104778.
- Ravi Sethi. *Programming Languages: Concepts and Constructs*. Addison-Wesley, Reading, Mass., USA, 2 edition, 1996.

*Note: The 3<sup>rd</sup> edition of Scott [2015], available as an Indian reprint, is also sufficient for the purpose. Wherever it is deficient, we will provide compensatory material.*